

MEV 저항을 위한 프로토콜 설계

이준서



목차

- MEV 정의 및 영향
- MEV 대응
 - Ethereum의 PBS
 - L0: Accountable Mempool
 - Monad의 MonadBFT

MEV 정의 및 영향

MEV 정의

➤ MEV(Miner Extractable Value)

- 채굴자가 블록 생성 시 트랜잭션 순서/포함 여부를 조작하여 얻는 이익
- e.g. 누군가의 거래 앞에 자신의 거래를 끼워 넣어 수익을 얻음

➤ MEV(Maximal Extractable Value)

- 블록을 제안하거나 구성하는 모든 주체가 트랜잭션 조작을 통해 얻을 수 있는 최대 이익
 - 참여자(Builder, Searcher, Validator 등) 다양해짐
 - 차익거래, 수수료 탈취 등

MEV 유형

- **Frontrunning**
 - 타겟 트랜잭션보다 MEV bot의 트랜잭션이 먼저 실행되게끔 함
 - 기회를 가로채거나 타겟 트랜잭션에 손실을 줄 수 있음
- **Backrunning**
 - 타겟 트랜잭션 이후에 결과 따라가며 이익을 얻는 전략
 - e.g. 차익거래 또는 DEX에서 유동성 회복될 때 바로 선점
- **Sandwich Attack**
 - 특정 트랜잭션 앞에 먼저 매수 주문하고 발생 직 후에 다시 매도를 하여 수익을 가져감
- **Non-Broadcast Transactions**
 - 트랜잭션을 다른 노드에 전파하지 않고 곧바로 자신이 생성할 블록에 임의대로 포함

MEV의 영향

- 네트워크에서 추출할 수 있는 가치를 극대화하는 ‘행위’
 - 공격으로 간주하면 안됨
 - 채굴자가 MEV 가치 추출하고 수익 극대화하는 것이 잘못된 행위는 아님
 - e.g. 차익거래는 시장의 건전성 유지 측면에서 긍정적일 수도 있음
 - 하지만 서비스 관점에서는 부정적
 - L1, L2 설계할 때 MEV 염두에 두고 설계를 해야 함
- 트랜잭션 검열
 - 특정 트랜잭션을 의도적으로 블록에 포함하지 않거나 지연시키는 행위

MEV 대표적인 대응 예시

- Ethereum의 PBS(Proposer Builder Seperation)
 - MEV 막지 않고 시장화해서 독점 방지
- Encrypted Mempool
 - 트랜잭션 내용을 암호화해서 내용 사전 감지하지 못하게 함
- L0: Accountable Mempool
 - 트랜잭션 선택, 정렬, 제외 등 모든 조작을 책임지고 추적
- Fair Ordering
 - 트랜잭션을 객관적 기준으로 정렬
 - 도착 시간, 무작위 등
 - MEV목적의 순서 조작 불가능

PBS: MEV의 탈중앙화



PBS 정의

- 블록 제안자(Proposer)과 블록 생성자(Builder)의 역할을 분리한 블록체인 아키텍처
 - 블록제안: Validator 승인을 위해 트랜잭션으로 구성된 블록을 제출하는 작업
 - 블록생성: Mempool에서 트랜잭션 선별하여 블록을 만드는 작업
- 기존 구조에서 Proposer의 권한을 줄여 MEV 수익을 전체 생태계에 분산

PBS 원리

- 블록 생성 단계를 두 개의 주체로 분리하여 MEV의 중앙화와 부작용 완화
 - **Proposer:** 블록 생성 최종적으로 결정하고 제안
 - 거래 직접 선별하거나 순서 정하지 않음. Builder가 제공한 블록 선택하는 역할
 - **Builder:** 최적화된 트랜잭션 묶음을 생성하고 Proposer에게 경쟁적으로 제안
 - MEV 추출할 수 있는 권한 가짐
 - 최대한의 수익성을 위해 경쟁적으로 블록 생성
 - MEV의 독점성 깨지고 투명한 경쟁시장 형성

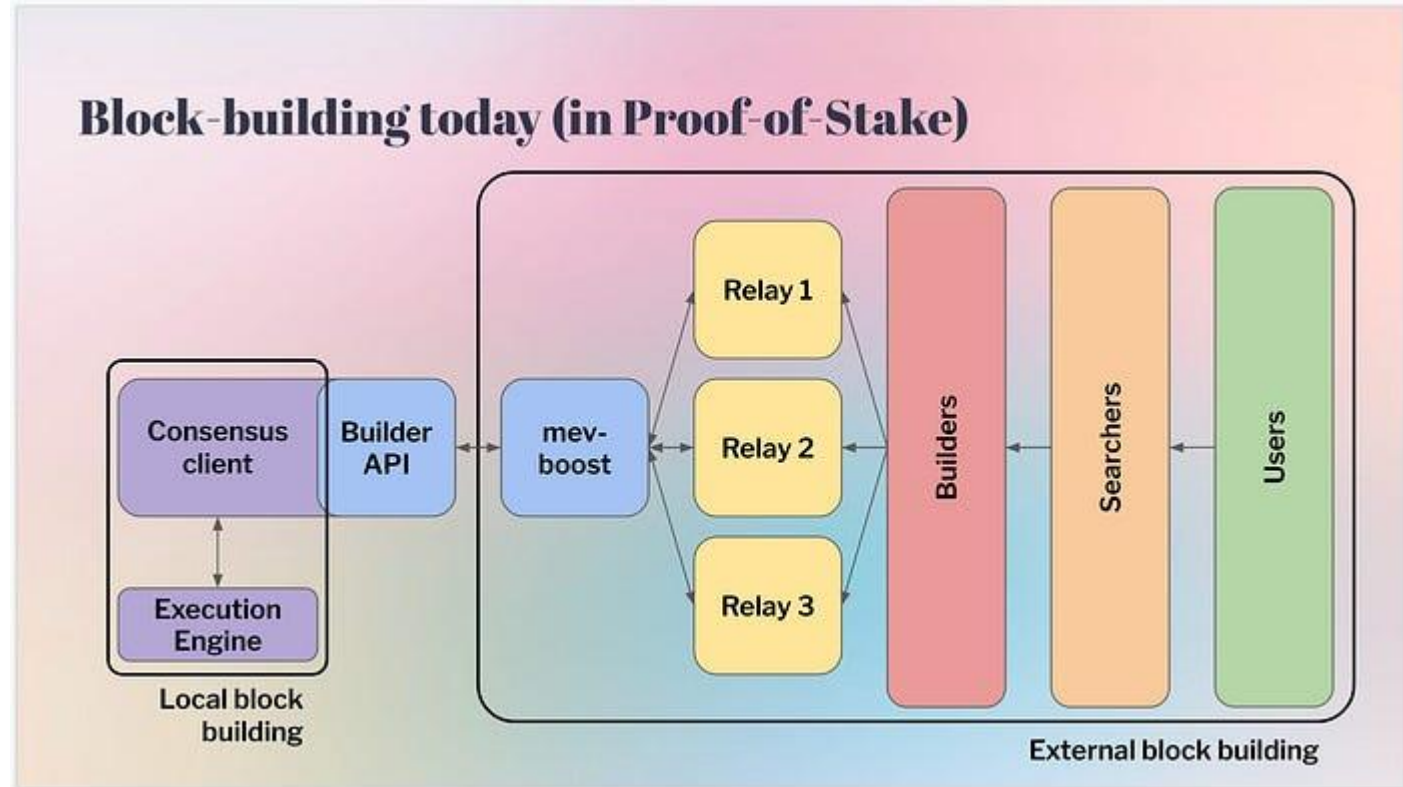
- PBS 동작 흐름
 - Builder는 거래 수집해서 MEV 최적화된 블록 구성
 - 자신이 만든 블록과 함께 거래 수수료 & MEV 수익 일부를 bid(입찰)로 Proposer에게 제시
 - Proposer는 여러 Builder가 제안한 블록 중 bid 가장 높은 블록 선택해서 네트워크에 전파

MEV-Boost

- PBS가 이더리움 in protocol로 구현되지 않았음
- 현재는 Builder와 Proposer를 연결하는 Middleware 형태의 MEV-boost로 구현
 - Proposer가 Builder의 블록을 쉽게 선택하고 받을 수 있도록 하는 도구
 - 여러 회사가 각자의 알고리즘으로 경쟁

블록 생성 과정

- Searchers
 - MEV 기회 탐지하고 거래 묶음
 - 수익 가능한 블록 설계
- Builders
 - 블록 구성
 - 트랜잭션 선택 및 정렬
- Relay
 - 여러 Builder 블록 수집
 - Proposer에게 전달



<Post Merge 블록 빌딩>

MEV 수익의 탈중앙화

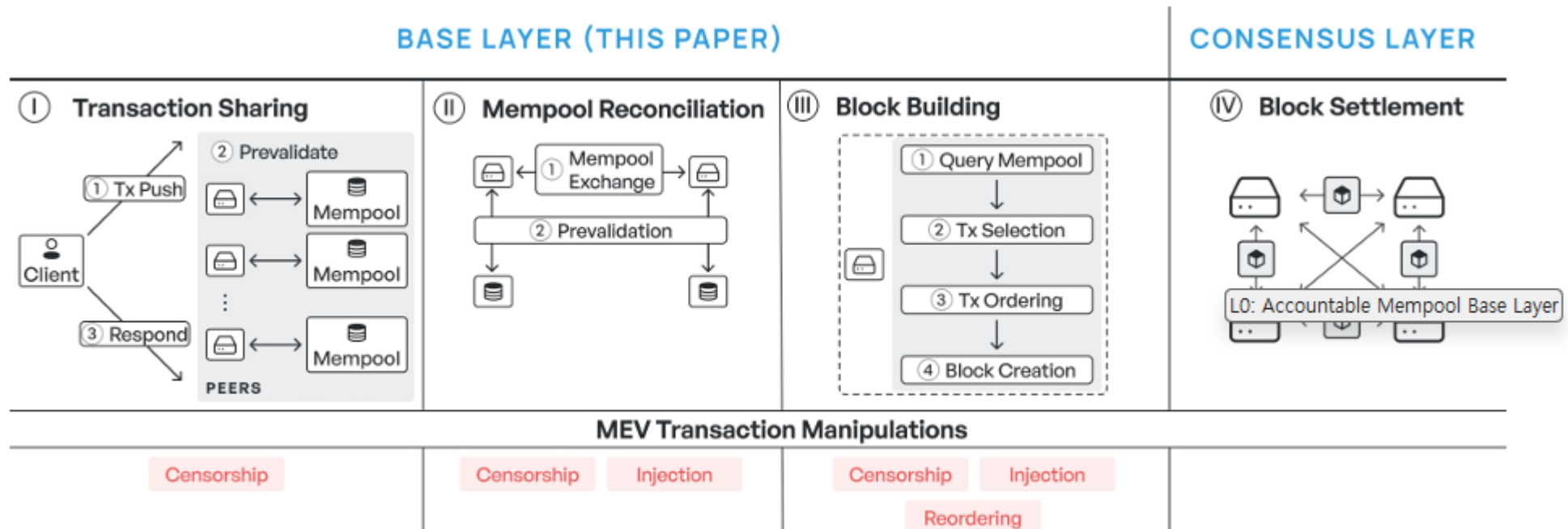
- MEV 추출에 특화된 Builder들이 MEV 추출하고 자신의 블록이 네트워크 추가될 수 있도록 경쟁하며 bid 로 Proposer에게 블록 전달
 - Proposer도 bid를 분배받아 MEV 수익 간접적으로 얻을 수 있음
 - MEV 수익이 전체적으로 탈중앙화 됨
- 개인 Validator도 MEV-Boost 도입하여 수익 분배받을 수 있음
 - 현재 90% 이상이 사용 중

L0: Accountable Mempool Base Layer



Base Layer에서 MEV 위험

- 초기 트랜잭션 공유
 - 악의적인 Peer의 검열 및 잘못된 트랜잭션 포함
- Mempool 조정
 - Peer간의 Mempool 공유하며 일부 트랜잭션 숨기거나 누락, 지연
- 블록 생성 (대부분의 MEV 발생)
 - 트랜잭션 주입, 재정렬, 블록 공간 검열



PBS와 다른 관점에서의 MEV 대응

- PBS는 수익의 분배 구조를 바꾸지만 MEV 자체를 제거하지 않음
- L0는 MEV 조작을 탐지하고 책임 추적 가능하도록 하는 Mempool 프로토콜
 - Base Layer에서 Batch의 트랜잭션 선택과 정렬 과정을 검증 가능하게 함
 - Base Layer 정의
 - ✓ 트랜잭션을 포함할지 여부를 결정하는 합의 이전 단계
 - Base Layer 역할
 - ✓ 트랜잭션 공유
 - ✓ Mempool 조정
 - ✓ 블록 생성
 - 검증자들은 Mempool 내용을 서로 교환하고 Commitment 기록
 - 자신이 관찰한 트랜잭션 및 시점을 허위로 보고하는 것을 불가능하게 함

Accountable Mempool L0

➤ Accountability 정의

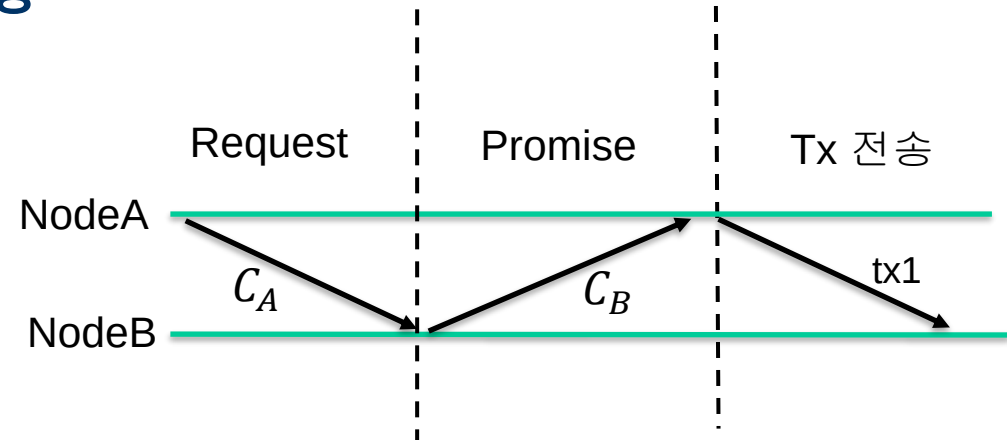
- 블록 제안자가 거래 순서 조작하거나 은폐하는 행위를 서로 감시하고 책임을 묻음
 - 받은 거래를 무시하거나 순서 조작할 경우, 그에 대한 책임 따름

➤ L0의 Mempool 공유 정책

- 모든 트랜잭션 포함
 - 모든 채굴자가 자신이 받은 트랜잭션을 Local Mempool로 유지하고 연결된 노드와 Mempool 동기화
- 수신 순서에 따른 트랜잭션 선택
 - Mempool에 저장된 순서대로 트랜잭션을 처리해야함
 - ✓ 각 노드의 수신 트랜잭션 순서는 다를 수 있음
- 검증 가능한 표준 블록 순서
 - Batch 내부의 거래 순서는 검증 가능한 랜덤 함수로 결정됨

Mempool Lifecycle

- 1. 노드 A에서 트랜잭션 tx1 생성되면 로컬 Commitment C_A 에 포함
 - Tx1 포함한 Batch 내의 트랜잭션들은 검증 가능한 랜덤 순서
 - 2. C_A 를 다른 노드들에게 전파 (Sync를 위해)
 - 3. 노드 B는 C_A 를 받으면 C_B 에 추가하고 응답
 - Batch 끼리의 트랜잭션 순서는 수신 받은 순서
 - 응답 안하면 의심, 응답할 때까지 요청
 - 4. 응답하면 A가 tx1포함 Batch를 B에게 전송
-
- Batch 내부 트랜잭션 랜덤 순서
 - Batch끼리는 수신 받은 순서
 - 동기화되지 않음



L0의 MEV 대응

➤ Censorship

- Mempool에서 공유 받은 트랜잭션은 무조건 포함시켜야해서 검열 불가능

➤ Injection

- 공유 받은 트랜잭션 모두 포함시키고 공유하지 않은 나만의 트랜잭션 포함시킬 수 있음
- 악의적인 Sandwich Attack 및 Frontrunning 불가능

➤ Reordering

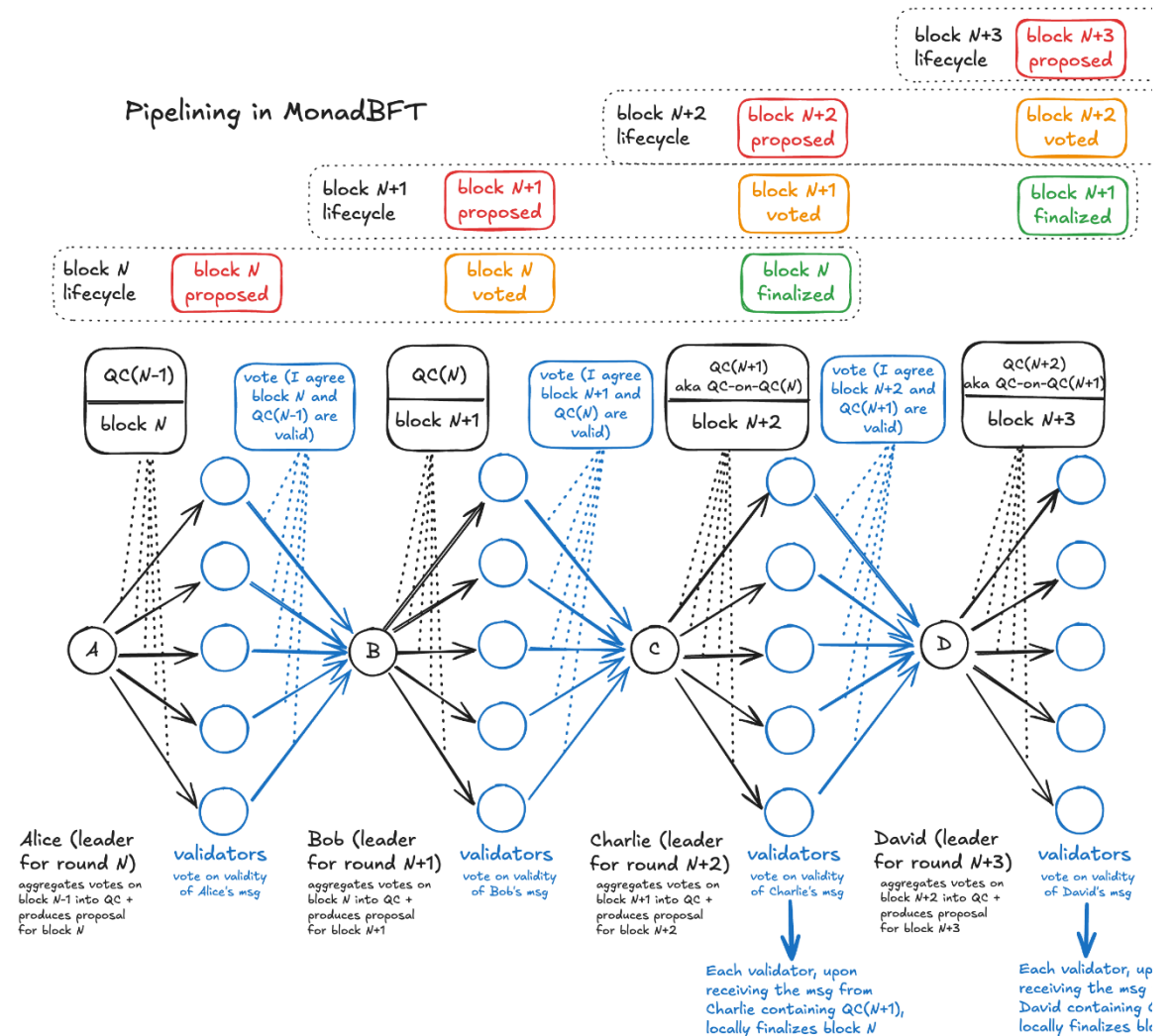
- Batch 내/외부 순서는 결정론적으로 정해지기 때문에 순서 조작 불가능

MonadBFT : Tail-forking Attack 방지



MonadBFT Overview

- Hotstuff 기반의 Chained BFT 합의 알고리즘
- Fast-Hotstuff 개선
 - 정상적인 리더 상황에서 안정적인 Happy Path
 - single-round speculatively
 - two-round finalize
 - 결함 있는 리더 상황에서 회복 경로 Unhappy Path
- Tail-forking 방지 설계
 - Hotstuff류 프로토콜의 고질적인 문제점
 - 다음 라운드의 리더가 이전 라운드의 리더의 블록을 체인에 추가하기 위해 투표 수집
 - 악의적인 리더의 악용 가능성 존재



Tail-forking 정의 및 MEV 취약점

➤ Tail-forking 정의

- 공격자가 의도적으로 이전 리더가 만든 블록을 버리고 자신의 블록을 제안하는 악의적인 Fork 공격
 - 합의 과정에서 일어날 수 있는 MEV 공격
- L3가 의도적으로 Fork하여 L2의 블록 버리고 자신의 블록을 제안

➤ Tail-forking의 MEV 취약점

- L2 블록의 내용을 탈취하여 무효화하고 자신의 블록으로 제안 가능
 - L2내용 재정렬 가능
 - 자신의 트랜잭션 Injection 가능
- L2는 블록 보상을 받지 못하게 됨

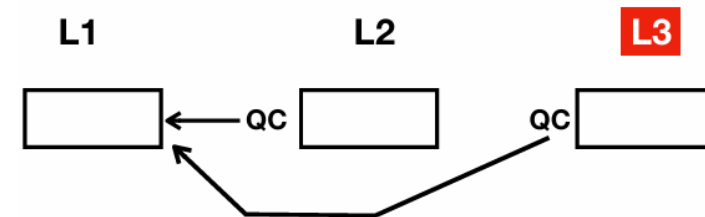
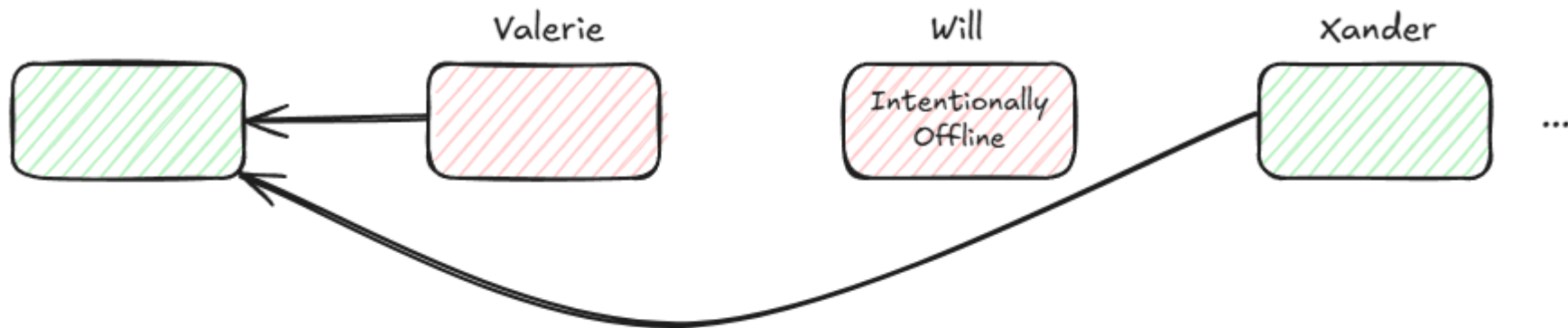


Fig. 1. Tail-Forking attack by the leader L3

Tail-forking 예시

- 1. Valerie가 블록 제안해 $2f+1$ 투표 받았음(QC 모았음)
- 2. Will이 악의적으로 오프라인인 척 타임아웃 유도
- 3. 다음 리더 Xander가 Valerie의 블록을 다시 제안할 수 있는 기회 생김
- 4. Valerie가 포함했던 고수익 거래, 차익거래 기회를 Xander가 자신의 이익 극대화되도록 재구성한 후 블록을 제출



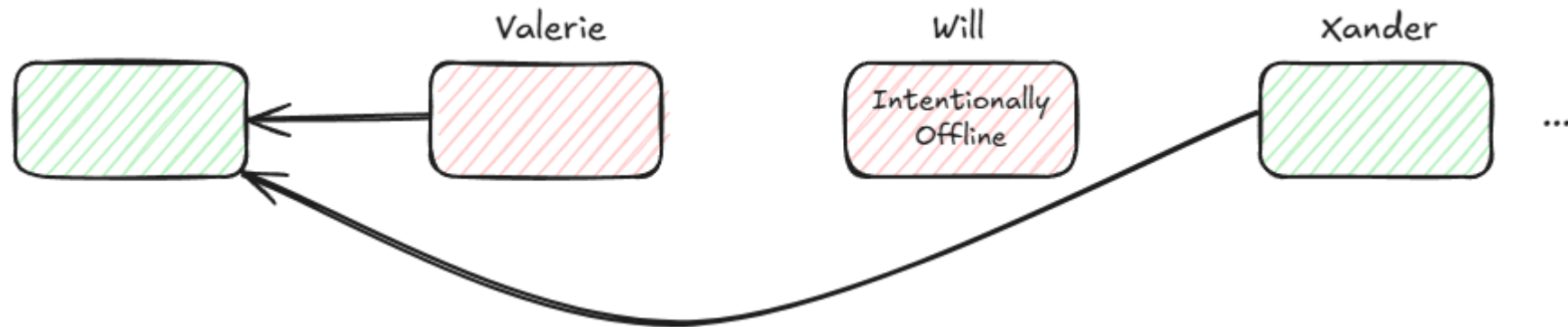
Will colludes with Xander and misses his slot, which invalidates Valerie's proposal. Xander can then propose a block inclusive of all available MEV opportunities for the past three rounds (from Valerie to Xander).

MonadBFT의 Tail-forking 대응

- 한 번 QC를 모은 블록은 최종적으로 확정되고 절대 폐기되지 않음
 - Paxos의 아이디어와 유사
 - 한번 제안되어 투표 받은 제안은 해당 Sequence에서 계속 지지
 - 추가 메타데이터와 강제 재제안 규칙이 합의 프로토콜에 추가되었음
- 강제 재제안 규칙
 - 다음 리더는 TC를 모은 경우 그 안에 담긴 가장 최근 투표된 블록을 반드시 재제안
 - 이전 블록 포함하지 않으려면 이전 블록이 $2f+1$ 투표 받지 못함을 증명 제시해야 함
 - NEC(No-Endorsement Certificate)

Tail-forking 대응 예시

- 1. Valerie는 $2f+1$ 투표 받았음
 - 2. Will이 오프라인이 되어 Valerie의 블록 보류되더라도
 - 3. Xander는 Valerie의 블록을 재제안하고 최종 포함시켜야 자신의 새로운 블록 제출 가능
-
- Tail-forking으로 인한 MEV 추출 차단



결론

- Mempool, 블록 생성, 합의 과정에서 MEV 대응에 대해 알아보았음
- Sui는 Mempool에 트랜잭션이 오래 머무르다보니 MEV 취약점이 이슈가 되기도 했음
- MEV는 Application Layer에서 막기에는 한계가 있어 아키텍처 전반적인 대응이 필요
 - Monad에서도 아키텍처 설계에서부터 MEV에 신경을 쓴 것처럼 보임
 - 이제 나올 L1 체인들의 키워드 중 하나는 MEV가 되지 않을까...

PBS 등장 배경

- 대부분의 Layer1 블록체인에서는 단일 검증인이 블록제안, 블록생성
- MEV로 인해 어떤 트랜잭션 포함할지 결정하고 자신의 트랜잭션 포함할지에 대한 전략 복잡
 - Validator 운영하는데 높은 고정 비용 발생
 - 소수의 Validator에게 유리한 환경 만들어짐
- PBS를 통해 검열 저항성 뿐만 아니라 Validator 진입 장벽 낮춤
 - 이더리움 네트워크에 다양한 네트워크 참여자 모을 수 있음
 - 탈중앙화를 이루기 위함

Encrypted Mempool의 한계

- Encrypted Mempool을 통한 MEV 대응
 - 트랜잭션을 Mempool에 제출할 때, 내용을 암호화해서 보냄
 - Proposer는 암호화된 상태에서 트랜잭션을 블록에 포함시키고 블록 커밋되거나 일정 조건 만족되면 복호화 키 공개
- Encrypted Mempool의 한계점
 - 암호화 및 복호화 Overhead
 - 실시간 UX 나빠짐
 - DoS 취약성
 - 의도적으로 많은 암호화된 트랜잭션 제출 시 시스템 과부하 가능
 - ✓ 암호화되어 있어 검증 불가능